

SIMATS ENGINEERING
CO 4: ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Operating Systems

COURSE CODE: CSA0408

COURSE OUTCOME COVERED: *co4: Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Course Outcome 1 Weightage: 20%

Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 5

Total Marks:25

Annexure A

INDUSTRY PROBLEM SOLVING TASK

Code of Conduct:

I D.NAVEEN KUMAR(192511401) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

D.NAVEEN KUMAR

Signature of the student:

Annexure A

INDUSTRY PROBLEM SOLVING TASK

S.No	Industry Problem Solving Task Questions	Marks
1	A cloud storage company is facing performance issues due to frequent file access delays. Analyze whether contiguous, linked, or indexed allocation would be most suitable for improving access speed and justify your choice with reasons.	5
2	A data center experiences high disk head movement while processing user requests. Suggest the most appropriate disk scheduling algorithm (FCFS, SSTF, SCAN) to reduce seektime and explain how it improves performance in this scenario.	5
3	An operating system uses a bitmap for free space management but struggles with slow allocation when the disk size increases. Propose an optimization technique to improve the efficiency of bitmap-based allocation and justify your solution.	5
4	A real-time application requires consistent and predictable disk access times. Evaluate whether SCAN or C-SCAN would be more suitable and explain your reasoning based on system requirements.	5
5	A company wants to design a hybrid storage system combining file allocation and disk scheduling techniques. Suggest a suitable combination (allocation + scheduling) for handling large-scale data access efficiently and justify your choice based on performance factors.	5

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Total
Conceptual Understanding	30%					
Accuracy of Answers	25%					
Application of Knowledge	20%					
Time Management	15%					
Question Interpretation	10%					
Total						

1. Best File Allocation Method for a Cloud Storage Company

A cloud storage company is experiencing delays because files are accessed frequently. Among **contiguous, linked, and indexed allocation**, the most suitable method is generally **Indexed Allocation**.

Analysis

Method	Access Speed	Random Access	File Growth	Main Limitation
Contiguous	Very high	Excellent	Difficult	External fragmentation
Linked	Low to moderate	Poor	Easy	Pointer traversal
Indexed	High	Excellent	Easy	Index-block overhead

Although **contiguous allocation** gives the fastest access because blocks are stored next to each other, it is difficult to maintain in a cloud environment where files are frequently created, deleted, and resized. It also causes external fragmentation.

Linked allocation avoids external fragmentation, but accessing a particular block requires following pointers through previous blocks, which creates delays.

Recommended: Indexed Allocation

In indexed allocation, an index block contains addresses of all data blocks belonging to the file.

Example:

```
Index Block
├── Block 120
├── Block 450
├── Block 680
└── Block 900
```

The operating system can directly find the required block without traversing all previous blocks.

Why it is suitable

- Supports fast **random access**.
- Suitable for frequently accessed files.

- Files can grow dynamically.
- No external fragmentation.
- Efficient for large files.
- Multilevel indexing can support extremely large files.

Conclusion: Indexed allocation is the best practical choice for cloud storage because it provides fast direct access while allowing flexible file growth.

2. Disk Scheduling for a Data Center with High Head Movement

A data center experiences excessive disk-head movement while processing a large number of requests.

Among **FCFS**, **SSTF**, and **SCAN**, **SSTF** or **SCAN** can reduce head movement, but for a busy data center, **SCAN** is usually the better overall choice.

Comparison

Algorithm	Seek Time	Head Movement	Fairness	Starvation
FCFS	High	High	Excellent	No
SSTF	Very low average	Very low	Poor	Possible
SCAN	Low	Controlled	Good	Very unlikely

Problem with FCFS

Suppose the head is at track 50 and requests are:

180, 20, 170, 30, 160

FCFS could cause:

50 → 180 → 20 → 170 → 30 → 160

This creates very large repeated head movements.

SCAN Approach

SCAN moves the disk head continuously in one direction while servicing requests.

For example:

50 → 160 → 170 → 180

Then reverses:

180 → 30 → 20

This avoids unnecessary back-and-forth movement.

Why SCAN is recommended

- Reduces excessive disk-head movement.
- Provides lower average seek time than FCFS.
- Handles heavy workloads efficiently.
- Provides better fairness than SSTF.
- Avoids the starvation problem associated with SSTF.
- Suitable when many disk requests arrive continuously.

Conclusion: SCAN is the most appropriate for a heavily loaded data center because it reduces seek time while maintaining fairness and stable performance.

3. Optimizing Bitmap-Based Free Space Management

In bitmap free-space management, every disk block is represented using a bit.

For example:

1 1 0 0 1 0 0 0 1

Where:

- 1 = allocated block
- 0 = free block

As disk size increases, the bitmap also becomes very large. Searching the entire bitmap for free blocks may therefore become slow.

Proposed Optimization: Hierarchical Bitmap

A good solution is to use a **hierarchical or multilevel bitmap**.

Instead of searching the complete bitmap every time, the system maintains a smaller summary bitmap that indicates which regions contain free blocks.

Example:

Level 1 Summary
1 0 1 0

↓

Region 1 → Has free blocks
Region 2 → Full
Region 3 → Has free blocks
Region 4 → Full

The operating system first checks the smaller summary bitmap.

If Region 3 contains free space, only that part of the main bitmap is searched.

Advantages

- Reduces bitmap scanning time.
- Faster free-block identification.
- Suitable for very large disks.
- Reduces CPU overhead.
- Makes allocation scalable.

Another optimization is to cache recently discovered free regions in memory so the operating system does not repeatedly scan the complete bitmap.

Example

Assume a disk contains:

1,000,000 blocks

Instead of searching all 1,000,000 bits, the system may divide the disk into 1,000 regions.

First:

Search 1,000 summary entries

Then:

Search only one selected region

This can significantly reduce allocation time.

Conclusion: A **hierarchical bitmap combined with caching of free regions** is an effective optimization for large disks because it avoids scanning the complete bitmap for every allocation.

4. SCAN vs C-SCAN for a Real-Time Application

A real-time application requires **consistent and predictable disk-access time**.

Between SCAN and C-SCAN, **C-SCAN is more suitable**.

SCAN

SCAN moves like an elevator.

0 → → → → 199
199 ← ← ← ← 0

Requests are serviced in both directions.

The waiting time of a request depends partly on where the disk head is and which direction it is currently travelling.

C-SCAN

C-SCAN services requests only in one direction.

0 → → → → → 199
 |
 ↓
0 ←----- 199
|
→ Start again

When the head reaches the end, it returns to the beginning and starts servicing requests again.

Comparison

Factor	SCAN	C-SCAN
Servicing direction	Both	One
Average seek time	Good	Good
Waiting-time uniformity	Moderate	High
Predictability	Moderate	Better
Fairness	Good	Very good
Real-time suitability	Good	Better of the two

Why C-SCAN is preferable

In SCAN, a newly arrived request may be serviced quickly or may wait for the head to travel all the way in the opposite direction.

C-SCAN provides a more uniform service pattern because all requests are approached in the same direction.

Therefore it gives:

- More consistent waiting times.
- Better predictability.
- Fairer treatment of tracks.
- Reduced variation in response time.

Important Note

For a **hard real-time system**, where missing a disk deadline is unacceptable, neither SCAN nor C-SCAN guarantees deadlines by itself. Deadline-based scheduling algorithms may be required.

Conclusion: Between the given options, **C-SCAN is more suitable for real-time applications because it provides more uniform and predictable disk-access times.**

5. Hybrid Storage System: File Allocation + Disk Scheduling

For a company handling very large amounts of data, a suitable hybrid combination is:

Indexed Allocation + SCAN Disk Scheduling

This combination provides a balance between fast file access and efficient disk-head movement.

Part 1: Indexed Allocation

Indexed allocation allows the operating system to directly locate file blocks using an index.

File	Index
—	Block 40
—	Block 120
—	Block 450
—	Block 700

Advantages:

- Fast random access.
- Supports large files.
- Easy file growth.
- No external fragmentation.
- Suitable for databases, cloud storage and large datasets.

Part 2: SCAN Scheduling

Once the required blocks are identified, SCAN determines the efficient order in which physical disk requests should be processed.

Example requests:

20, 40, 85, 110, 160, 180

If the head is at 70 and moving upward:

70 → 85 → 110 → 160 → 180

Then it reverses:

180 → 40 → 20

This reduces unnecessary head movement.

Why the combination works

Requirement	Technique	Benefit
Fast file-block location	Indexed allocation	Direct/random access
Large file support	Indexed allocation	Multilevel indexing
Reduce fragmentation	Indexed allocation	Blocks need not be contiguous
Reduce seek time	SCAN	Organized head movement
Heavy workload handling	SCAN	Good throughput
Fair disk service	SCAN	Low starvation risk

How the Hybrid System Works

User requests file
↓
Indexed Allocation
locates required blocks
↓
Disk requests generated
↓
SCAN Scheduler
orders requests efficiently
↓
Disk reads/writes blocks
↓
Data returned to user

Example

Suppose a large file is stored in:

Block 45, 110, 160, 250, 400

Indexed allocation immediately identifies these blocks.

SCAN then determines an efficient order for accessing them instead of moving randomly between locations.

This provides:

Fast block lookup + Reduced head movement + High throughput

Final Recommendation

Indexed Allocation + SCAN Disk Scheduling is a strong combination for large-scale storage systems because indexed allocation provides flexible and fast file access, while SCAN reduces physical disk seek time and handles heavy workloads fairly.